HOMEWORK POLICY
IISc Mech. Eng., Applied Dynamics I
Homework

Date of this version: August 21, 2023

Hyperlinks look like this.

1 Note to homework grader(s)

Dear graders and gradees:

As a grader please grade homework like this. As a student, please keep these things in mind. Read this policy once per week until you have digested the whole of it. Let us know if anything is unclear, needs improvement, should be changed substantially or should be added.

1. **Rubric.** If peer grading is used, the peer graders are not giving a grade or a score, but rather a critique. You should be giving the most useful feedback possible to both the student, and the TA who will be giving a grade. The peer grader should keep in mind the following. However, you do not have to answer these questions literally.

2. **Could the student you are grading do the problem with infinite time?** What is the probability, out of 100%, as best you can assess it from the homework you are looking at, of the following. Imagine the given student is on an island with no help and no access to GOOGLE. They are given a similar problem to do. Someone cares about the result but is suspicious. The student has 2 weeks to do the problem. That is, there is no issue of having enough time. *What is the probability* that they could get the right answer and convincingly write it up?

3. **Imagine you care.** Imagine that you are a manager at the great new ambitious fast-growing company AUTONOMOUS-ELECTRO-TUBE Y^2. You are supposed to design the next energy-blasting hyper app. You need to hire a Cornell Engineer to do something that is based on the skills from the homework you are now grading. How much, based on the homework before you, would you trust the student to do the job (assuming they have no outside help but have plenty of time).

4. **Mastery.** Does the student have mastery of the problem? This could be really deep. In the extreme, for example, you might think that a student should get a full homework grade and extra credit, even if they did none of the other problems in the set. A great student effort is not necessarily partitioned evenly amongst the assigned problems, a great homework might even have some poorly done or missing problems.

5. **Clarity.** Consistently with the things above, check for clear communication including.

   (a) On the first page of the homework, check for the following, to ease your sorting:

i. On the top right corner look for student's name, course # and name, due date.

ii. At the top, students should write how many hours they worked on the homework. We need this so we can separate their effort level from their skill level. This also helps us gauge the work load we are imposing.

iii. **Student's should cite their help.** Students are allowed to get any amount of help from any sources or people. At the top of each problem they should acknowledge all help they got from, e.g., TAs, faculty, students, or any other source. They could write, for example: "Mitch McConnel pointed out to me that I needed to draw the second FBD in problem 2." or "Susan Collins showed me how to do problem 3 from start to finish." or "I copied this solution word for word from Ruth Ginsburg " or "I found a problem just like this one, number 386.5.6, at boydowehavesolutionsforyou.com, and copied it." etc. Still, they should follow academic integrity principles: they must be clear about which parts of their presentation they did not do on their own. It is hereby declared that violation of this policy is not a violation of 'Academic integrity' rules. It is, however, a violation of academic integrity. So here, where it is safe, you can note and encourage that as you grade. But do not take off credit for suspected violations.

(b) **FBDs**. *Every* use of linear or angular momentum balance must be associated with a **clear correct free body diagram**.

(c) **Vector notation** must be clear and correct (As described in, *e.g.,* Ruina and Pratap).

(d) **Units.** Every line of every calculation should be dimensionally correct (students should carry units as described in the Ruina/Pratap book appendix, and as seen in the examples therein).

(e) **Clear layout.** Work should be laid out neatly enough to be read by someone who does not know how to do the problem. Part of their jobs as an engineers will be to *convincingly* get the right answers. Their job on the homework is to practice this. They should strive for "solutions quality". Each problem should start on a fresh page; a new problem should not start on the same page as the ending of another problem.

(f) **Self contained** A solution should be self-contained, including, for example, enough of a problem restatement so that *a reader can understand the solution without ever seeing the posted problem statement.* Solutions should be clear and convincing enough so that another student (who has not done the problem and does not know how to do it) can read the solution, understand it, and judge that it is correct.

(g) **Boxed or circled answers.** Answers and main results should be clearly marked.

(h) **Posted solutions.** Look out for student work that is complete and clear enough that we can use it as an example to post.

(i) **Make work: 'Can do, don't want to.'** In general, that is always, students should be doing problems because doing those problems are educational. Some problems may seem like make-work to some students because they already knows

how to do them. If so, the TAs will give 95% credit if they write, in full "I can do this problem but don't feel I will gain from writing out the solution" or, in short, "Can do, don't want to." But, they have to be honest. Let them know if you doubt their self-assessment. For example "Can do, don't want to" should not be confused with "Could look up, don't want to.". Or, a student might not know how to do a problem. But, they think they can learn better by spending time on a different problem. They should say so and do a different problem. That's fine.

(j) **Regrading.** So long as they have made a good attempt at their first effort, students can get 80% credit by redoing a problem. These will be looked at in a pile at the end of the semester.

(k) **Binder.** The students should be keeping a binder (or folder or notebook) of all of their work. At the end of the semester we will ask the students to show a folder of their completed HW. Thus, at the end we have a chance to assess the net homework effort and quality.

(l) **MATLAB, Computer work**

i. **Clear and identified.** The student's name should be near the top of the computer text file. Student's should highlight (or circle with a colored pen) their name on the computer printout. At least some part of any other computer output should also include the student's name, printed by the computer. They should highlight (or circle) their name on each page. We are not penalizing copying, but we do not want to facilitate it.

ii. **Commented.** Code should be well commented.

iii. **Derivations.** Student's should show hand calculations for derivation of equations of motion. Or, equivalently, show clear computer algebra inputs and outputs.

iv. **Print out.** They should print and attach the entirety of the code they use, with additional hand-written annotations, as is needed to make the code easy to read.

v. **Verify.** Students should show, by whatever combination of methods they have the ability and time for, the reasons they think their solution is correct. Mastering these things is the key to debugging. Some ideas:

A. It matches an analytic solution.

B. One or more special cases match an analytic solution.

C. Obeys one or another known and applicable symmetry or conservation law.

D. It matches intuitive trends.

Use figures and numerical values to show these consistencies between the numerical solution and things that are known or suspected to be true.

vi. **Answer the question.** Work should show the answer clearly. Student work should, for each problem, earn your trust. Often, figures will show the result, and need annotation to point it out. And an explanation in words. And, if appropriate, numbers. Arrows and circles and annotations can greatly clarify a computer's plots or numerical output.

vii. **Labeled plots.** Each plot needs both axis labeled and a title. Computer labeling is fine, but it is also fine to label computer plots by hand.

viii. `axis equal`. If both axis represent physical position, unless you have good reason to do otherwise, use Matlab's "`axis equal`". This way, circles will look round, squares square, and trajectories will have the right shapes.

ix. `axis ([-6 6 0 10])`. Use a command like this to keep your view constant as an animation proceeds. Without this, it's as if the camera is panning and zooming, following your object and not really showing its motion.

x. `Big Fonts`. Put this in your `startup.m` file and leave it there all semester: `set(0,'DefaultAxesFontSize', 20)`. Or, put a line like this in all of your `main.m` files. 25 or 30 is ok too.

xi. **Readable files, including right-hand-side files.** As a general rule, computer commands should not be written in hard-to-decypher code. Rather, each line should be readable and checkable. More short lines of code are usually easier to read, sometimes far easier, than long lines of code.

xii. In ODE problems, always look for the following.

A. **Use of clear variables.** Variables and constants should be unpacked into readable variables. No calculations should be done with, for example, `z(4)` and `z(7)`. Rather, formulas should use `theta1` and `v3` and such like. Using readable variables makes the code 117 times more readable and debuggable.

B. **Structs.** Pass parameters using structs. Namely p.m, p.G etc, not a list of variables. (However, as far as I know, you have to abandon this rule when using `matlabFunction`.)

C. **Don't throw in a random use of Euler's method.** If you are solving your ODEs with, say, ODE45, then also use ODE45 for your energy check by adding equations like `Wdot = P` to your list of ODEs.

D. **Black and white.** You can use color. But your plots need to make sense when printed in black and white. So use, say, *thick* red lines and *thin* blue lines.

E. **tspan.** Unless all you want is the solution at the final time, *always* use something like `tspan = linspace(0,10,1001)`, rather than `tspan=(0,10)`. This way, animations will not move in erratic ways due to uneven time steps. You get output at times you care about. **Exception:** you can, instead, use the simpler two-number version of `tspan` and interpolate the solution as needed using `soln = ode45(...)` and `deval(soln,t)`.)

F. **Options.** Appropriate use these features of ODESET: 'RelTol', 'AbsTol', 'events', etc.

G. Generally, don't invert matrices symbollically. If you don't understand this direction, do this experiment. Run these MATLAB commands to symbolically invert a 5×5 matrix:

```
syms a b c d e f g h i j k l m n o p q r s t u v w x y real
A = [a b c d e f g h i j k l m n o p q r s t u v w x y];
A = reshape(A,[5 5]);
inv(A)
```

You will get a three page answer. It's worse for 6×6 and bigger. Whereas, with numbers the inverse is just a matrix made up of the same number of numbers. Then run these MATLAB commands:

```
A = rand(5);
inv(A)
```